

Tugas Pertemuan 2

Kelompok, Domain, dan Spesifikasi Contract-First

Unit pengerjaan Kelompok (4 orang), disertai pernyataan kontribusi individu

Tag repositori 11 (Bagian A) · 12 (Bagian B)

Bobot Bagian dari 15% nilai laboratorium; menjadi dasar penyusunan A1 (25%)

0. Ketentuan umum

1. **Kode implementasi service tidak boleh ditulis dalam tugas ini.** Commit yang memuat kode service dengan tanggal lebih awal daripada commit spesifikasi dinilai FAIL untuk Bagian B. Ketentuan ini berlaku karena antarmuka yang disusun setelah implementasi cenderung mendeskripsikan implementasi tersebut — struktur tabel, konvensi framework, dan asumsi penulisnya — sehingga tidak dapat digunakan oleh konsumen lain yang dibangun dengan pendekatan berbeda. Pada Pertemuan 7, kelompok lain akan membangun klien berdasarkan dokumen ini tanpa komunikasi dengan penyusunnya.
2. **Setiap anggota melakukan commit menggunakan identitas Git masing-masing.** Commit dari akun bersama menyebabkan pernyataan kontribusi tidak dapat diverifikasi.
3. **Justifikasi desain harus dinyatakan dalam istilah kendala, bukan nama platform.** Bentuk "karena ini aplikasi mobile" tidak diberi nilai. Bentuk yang dinilai: "karena klien ini tidak dapat menyimpan rahasia dan jaringannya intermiten, klien ini memerlukan public-client flow dan durable mutation queue".

Bagian A — Kelompok, Domain, dan Taksonomi Klien

Jatuh tempo: akhir sesi Pertemuan 2 · **Tag repositori:** 11

A.1 Pembentukan kelompok

Kelompok terdiri atas empat orang dan bersifat tetap selama satu semester. Tetapkan empat peran berikut dan catat dalam `README.md`. Peran berotasi setiap tiga pertemuan; seorang anggota tidak boleh memegang peran yang sama dua periode berturut-turut.

PERAN	TANGGUNG JAWAB
Contract owner	Pemegang tanggung jawab atas <code>openapi.yaml</code> . Setiap perubahan antarmuka ditinjau oleh peran ini, terlepas dari siapa penulisnya.
Service owner	Backend yang di-deploy, konfigurasi, migrasi, dan health endpoint-nya (berlaku mulai Pertemuan 3).
Client owner	Klien yang dihadapi pengguna, serta pelaporan tertulis atas setiap ambiguitas yang ditemukan dalam kontrak.
Integration owner	Mock server, contract test, dan koordinasi dengan kelompok mitra pada Pertemuan 7.

Pembagian peran wajib didokumentasikan agar beban kerja terdistribusi dan setiap anggota memiliki tanggung jawab teknis yang dapat dinilai secara individual pada Pertemuan 14.

A.2 Penetapan domain

Susun satu paragraf prosa — bukan daftar fitur — yang menjelaskan proses yang terjadi dari sudut pandang pengguna sistem: siapa yang bertindak, dalam urutan apa, dan apa yang dapat gagal.

Batas cakupan: paragraf tersebut harus dapat dibacakan dalam waktu tiga puluh detik. Paragraf yang lebih panjang menandakan cakupan yang terlalu besar untuk diselesaikan sampai Pertemuan 14.

Domain wajib memenuhi keempat syarat berikut. Tuliskan hasil pemeriksaan secara eksplisit, satu baris untuk setiap syarat.

1. **Terdapat minimal tiga jenis aktor dengan hak akses yang berbeda.** Sistem dengan satu jenis pengguna tidak menyediakan bahan untuk materi Pertemuan 4.
2. **Terdapat minimal satu operasi yang tidak aman (*unsafe*) dan konsekuensial**, yaitu operasi yang tidak boleh terjadi dua kali. Transaksi keuangan merupakan contoh utama; pemesanan, pengumpulan tugas, dan perubahan status yang tidak dapat dibatalkan juga memenuhi syarat.
3. **Terdapat minimal satu aktor yang bekerja di luar jangkauan konektivitas yang andal.** Apabila seluruh pengguna berada di lokasi tetap, materi Pertemuan 6 tidak dapat diterapkan.
4. **Cakupan cukup kecil untuk diselesaikan**, yaitu satu alur kerja secara utuh dari awal sampai akhir.

Syarat yang tidak terpenuhi harus diperbaiki pada pertemuan ini dengan mengubah domain, tidak boleh ditunda.

A.3 Taksonomi lima sumbu

Susun tabel dengan satu baris untuk setiap klien yang direncanakan dan lima kolom untuk lima sumbu berikut. Seluruh sel harus terisi. Nilai "tidak diketahui" hanya berlaku untuk baris kelompok mitra.

SUMBU	PERTANYAAN	MENENTUKAN MATERI
1. Kemampuan menyimpan rahasia	Dapatkah kredensial disimpan di luar jangkauan pengguna dan pihak yang menguasai perangkat?	Pertemuan 4 (identitas)
2. Ketersediaan jaringan	Selalu tersedia, intermiten, atau sering tidak tersedia?	Pertemuan 2 (idempotency), Pertemuan 6 (offline)
3. Anggaran latensi	Berapa lama sebuah operasi boleh berlangsung sebelum tujuan klien tidak tercapai?	Pertemuan 10 (real-time)
4. Batas sumber daya	Memori, penyimpanan, daya, dan biaya bandwidth.	Pertemuan 11 (IoT)
5. Kehadiran manusia	Apakah ada manusia yang menafsirkan respons yang ambigu? Agen otonom akan menafsirkan secara mandiri dan langsung bertindak.	Pertemuan 2 (desain error), Pertemuan 12–13

Untuk setiap baris, tuliskan satu kalimat kesimpulan dengan bentuk berikut:

Karena jaringan aplikasi kurir sering tidak tersedia, klien tersebut memerlukan durable mutation queue, dan operasi unsafe pada sistem ini memerlukan idempotency key.

A.4 Dekomposisi aturan bisnis

Pilih satu aturan bisnis yang paling penting dalam domain, yaitu aturan yang pelanggaranannya menimbulkan dampak terbesar, kemudian dekomposisikan aturan tersebut ke dalam tiga lapisan berikut.

LAPISAN PERAN TERHADAP ATURAN

Service	Menegakkan aturan. Merupakan satu-satunya lapisan tempat aturan tersebut berlaku secara otoritatif.
Kontrak	Menyatakan aturan: transisi yang diizinkan, status code yang dikembalikan saat aturan dilanggar, dan field yang memungkinkan klien memprediksi hasil.
Klien	Memprediksi aturan, misalnya dengan menyembunyikan kontrol tertentu. Bersifat peningkatan pengalaman pengguna, bukan mekanisme kendali.

Aturan tersebut harus muncul tepat satu kali sebagai penegakan. Aturan yang ditulis sebagai penegakan di dua lapisan menandakan adanya cacat desain.

A.5 Struktur repositori

Struktur berikut wajib ada sejak commit pertama.

openapi.yaml	kontrak – berkas utama dalam repositori ini
CHANGELOG.md	catatan setiap perubahan kontrak, bertanggal
README.md	petunjuk menjalankan seluruh komponen
docs/	
decisions/	satu berkas untuk setiap keputusan signifikan
service/	backend (Pertemuan 3)
clients/	
web/	Pertemuan 5
mobile/	Pertemuan 6
device/	Pertemuan 11
mcp/	Pertemuan 12
tests/	
contract/	pengujian kesesuaian service terhadap openapi.yaml

```
git init nama-tim && cd nama-tim
mkdir -p docs/decisions service clients/{web,mobile,device,mcp} tests/contract
printf '# Contract changelog\n' > CHANGELOG.md
git add -A && git commit -m "Repository skeleton"
```

Setiap anggota melakukan clone dan membuat satu commit dengan identitasnya sendiri pada pertemuan ini. Verifikasi dilakukan dengan perintah `git log --format='%an' | sort -u`, yang harus menampilkan empat nama berbeda.

A.6 Pencatatan keputusan

Buat berkas `docs/decisions/0001-domain.md` dengan empat bagian: **Context**, **Decision**, **Alternatives considered**, dan **Consequences**. Catat pemilihan domain pada berkas tersebut. Praktik ini dilanjutkan sepanjang semester untuk setiap keputusan yang memerlukan penjelasan bagi pembaca di kemudian hari, dan menjadi bahan pertanggungjawaban pada Pertemuan 14.

Kriteria penilaian Bagian A

KATEGORI KRITERIA

PASS Paragraf domain tersedia; empat syarat diperiksa secara tertulis; tabel lima sumbu disertai satu kesimpulan per baris; satu aturan bisnis terdekomposisi; repositori berstruktur sesuai ketentuan dengan empat committer berbeda; pembagian peran tercatat.

PARTIAL Repositori dan domain tersedia, tetapi tabel taksonomi berupa daftar platform, bukan analisis kendala.

FAIL Repositori tidak tersedia, atau domain tidak memenuhi dua syarat atau lebih.

Bagian B — Spesifikasi Contract-First dan Mock Server

Jatuh tempo: sebelum Pertemuan 3 · **Tag repositori:** 12

Pekerjaan dilakukan terhadap domain kelompok masing-masing, bukan terhadap studi kasus KANTIN. Keluaran berupa satu berkas `openapi.yaml` di akar repositori yang mendeskripsikan antarmuka yang belum diimplementasikan.

B.1 Pemodelan resource

Resource adalah kata benda dalam domain yang memenuhi ketiga kriteria berikut.

1. **Identitas.** Entitas tersebut dapat ditunjuk dan diberi nama. Sebuah order memiliki identifier yang dapat ditempatkan dalam URI; "proses checkout" tidak memilikinya.
2. **Masa hidup.** Entitas tersebut tetap ada melintasi request. Order tetap ada setelah request yang membuatnya selesai; hasil validasi tidak.
3. **Kemandirian.** Entitas tersebut dapat berubah tanpa mengharuskan entitas lain dibuat ulang.

Ekstrak seluruh kata benda dari paragraf domain, kemudian uji setiap kandidat terhadap ketiga kriteria tersebut. Serahkan hasilnya dalam bentuk tabel dengan kolom **kandidat**, **keputusan**, dan **alasan**. Penilaian didasarkan pada kolom alasan; keputusan tanpa alasan tidak diberi nilai.

Tabel wajib memuat minimal dua kandidat yang ditolak beserta alasan penolakannya.

Ketentuan mengenai endpoint berbentuk layar. Endpoint tidak boleh dirancang berdasarkan tampilan antarmuka. Endpoint seperti `GET /home` yang mengembalikan beberapa jenis data sekaligus hanya sesuai untuk satu klien. Klien berikutnya akan memerlukan sebagian data tersebut atau bentuk data yang berbeda, sehingga service akan memiliki banyak endpoint dengan data yang saling tumpang tindih. Ketentuan yang berlaku: resource diturunkan dari domain, sedangkan tampilan merupakan komposisi resource yang dilakukan oleh klien.

B.2 Aturan desain URI

NO.	ATURAN	BENTUK YANG BENAR	BENTUK YANG SALAH
1	Menggunakan kata benda jamak, bukan kata kerja	<code>POST /v1/orders</code>	<code>POST /v1/createNewOrder</code>
2	Identitas pada path, penyaringan pada query	<code>GET /v1/orders? outletId=out_7Kq&status=placed</code>	<code>GET /v1/ordersByOutlet/out_7Kq</code>
3	Penyaringan maksimal satu tingkat	<code>/v1/deliveries/dlv_9Zx</code> yang memuat field <code>orderId</code>	<code>/v1/outlets/{a}/orders/{b}/ deliveries/{c}</code>
4	Identifier bersifat opaque dan dihasilkan server	<code>ord_51Fa93cD</code>	<code>/v1/orders/1</code>
5	Transisi status dimodelkan sebagai sub-resource	<code>POST /v1/orders/{id}/cancellation</code>	<code>POST /v1/cancelOrder</code>

Pengujian untuk aturan 2: penghapusan sebuah query parameter harus menghasilkan lebih banyak data dari jenis yang sama, bukan data dengan jenis berbeda. Apabila penghapusan parameter mengubah bentuk respons, parameter tersebut termasuk bagian dari path.

Aturan 4 berlaku karena identifier auto-increment mengungkapkan volume transaksi, memungkinkan enumerasi data, dan mengikat antarmuka pada satu basis data tertentu.

B.3 Safety dan idempotency

- Sebuah method bersifat **safe** apabila tidak mengubah state. Hanya **GET** yang bersifat safe.
- Sebuah method bersifat **idempotent** apabila pelaksanaan request identik dua kali menghasilkan state akhir yang sama. Kriterianya adalah state, bukan respons.

METHOD	SAFE	IDEMPOTENT	DAPAT DI-RETRY OLEH KLIEN YANG TERPUTUS
GET	Ya	Ya	Ya
PUT	Tidak	Ya	Ya
DELETE	Tidak	Ya	Ya; panggilan kedua mengembalikan 404 atau 204 dengan state yang sama
PATCH	Tidak	Tidak secara inheren	Hanya apabila patch bersifat absolut (<code>{"quantity": 3}</code>), bukan relatif (<code>{"quantity": "+1"}</code>)
POST	Tidak	Tidak	Tidak, kecuali disertai idempotency key

Prosedur pemeriksaan untuk setiap operasi: tentukan konsekuensi bagi pengguna apabila koneksi terputus dan klien mengirimkan request yang sama untuk kedua kalinya. Apabila tidak ada konsekuensi, operasi tersebut idempoten dan tidak memerlukan perlindungan tambahan. Apabila konsekuensinya berupa pembebanan biaya ganda, penggandaan data, atau pengiriman notifikasi kedua, operasi tersebut memerlukan idempotency key yang harus dispesifikasikan.

Empat ketentuan yang wajib dinyatakan dalam spesifikasi. Pernyataan bahwa sistem menggunakan idempotency key tanpa perincian berikut tidak diberi nilai.

1. **Nama header dan format nilainya.** Contoh: **Idempotency-Key**, berupa UUID versi 4 dalam bentuk kanonik dengan tanda hubung.
2. **Operasi yang mewajibkannya.** Contoh: wajib pada **POST /v1/orders** dan **POST /v1/orders/{id}/payment**; diabaikan pada operasi lain.
3. **Jendela retensi.** Contoh: key disimpan selama 24 jam; key yang digunakan kembali setelah periode tersebut diperlakukan sebagai key baru. Nilai ini diperlukan klien untuk menyusun kebijakan retry.
4. **Perilaku saat key digunakan kembali dengan body berbeda.** Contoh: mengembalikan **409 Conflict** dengan **type** bernilai `.../problems/idempotency-key-reuse`.

Perilaku server yang wajib dinyatakan dalam spesifikasi:

KONDISI	TINDAKAN SERVER	RESPONS KEPADA KLIEN
Key belum pernah diterima	Mencatat key, memproses request, dan menyimpan respons terhadap key tersebut	201 Created
Key sudah diterima, body identik	Tidak memproses ulang; mengirimkan kembali respons yang tersimpan	201 dengan identifier yang sama
Key sudah diterima, body berbeda	Menolak request karena key digunakan untuk maksud yang berbeda	409 Conflict disertai Problem Details
Key sudah diterima, request asal masih diproses	Tidak memproses dua kali; mengembalikan sinyal retry	409 disertai header Retry-After

Ketentuan pembangkitan key. Key dibangkitkan pada saat pengguna melakukan konfirmasi, disimpan bersama request yang tertunda, dan digunakan kembali sampai request tersebut berhasil. Key yang dibangkitkan di dalam fungsi pengiriman jaringan tidak memenuhi tujuannya, karena setiap percobaan ulang menghasilkan key baru dan server memperlakukannya sebagai request yang berbeda.

B.4 Status code dan katalog error

Terdapat tiga kategori kegagalan, yang ditentukan berdasarkan tindakan yang harus diambil klien.

KATEGORI	PERTANYAAN PENENTU	STATUS CODE	TINDAKAN KLIEN
Client fault	Apakah klien mengirimkan request yang tidak valid?	400 , 401 , 403 , 404 , 422	Tidak melakukan retry. Memperbaiki request atau meminta tindakan pengguna.
Domain rejection	Apakah request valid tetapi ditolak oleh aturan bisnis?	409 , 422	Tidak melakukan retry. Menampilkan alasan penolakan kepada pengguna. Kondisi ini merupakan hasil proses, bukan kesalahan sistem.
Server fault	Apakah kegagalan berasal dari service?	500 , 502 , 503 , 504	Melakukan retry dengan exponential backoff. Tidak menampilkan detail internal kepada pengguna.

Dua kesalahan yang umum terjadi pada kategori *domain rejection*:

- Mengembalikan 500 untuk penolakan aturan bisnis menyebabkan klien melakukan retry atas request yang tidak akan berhasil, dan mencampurkan peristiwa bisnis normal ke dalam pemantauan kesalahan sistem.
- Mengembalikan 200 disertai {"success": false} menyebabkan seluruh lapisan HTTP generik antara service dan klien — mekanisme retry, pemantauan, caching, dan penanganan error pada kelompok mitra — memperlakukan kegagalan tersebut sebagai keberhasilan.

Format error tunggal: RFC 9457 **Problem Details**, dengan Content-Type: application/problem+json .

FIELD	KETENTUAN
type	Wajib. URI stabil yang mengidentifikasi jenis masalah, dan menjadi dasar percabangan logika pada klien. Nilai ini tidak boleh diubah setelah dipublikasikan.
title	Ringkasan singkat untuk pembaca manusia. Bersifat tetap untuk setiap type .
status	Status code HTTP, diulang dalam body agar dokumen tetap bermakna ketika dicatat dalam log atau diteruskan.
detail	Keterangan spesifik untuk kejadian tersebut. Tidak boleh memuat stack trace, potongan kueri SQL, atau hostname internal.
instance	URI kejadian spesifik. Correlation id ditempatkan pada field ini mulai Pertemuan 9.
extension members	Field tambahan yang ditentukan sendiri. Bagian ini dinilai secara eksplisit pada A1.

Contoh penolakan domain secara lengkap:

```

HTTP/1.1 409 Conflict
Content-Type: application/problem+json

{
  "type": "https://api.contoh.example/problems/item-unavailable",
  "title": "A requested menu item is unavailable",
  "status": 409,
  "detail": "Ayam geprek (itm_3Bn) sold out at 12:14 local time.",
  "instance": "/v1/orders",
  "unavailableItems": [ "itm_3Bn" ],
  "suggestedAlternatives": [ "itm_9Lp", "itm_2Rt" ]
}

```

Tiga field terakhir merupakan *extension members* di luar ketentuan RFC. Field `unavailableItems` memungkinkan aplikasi klien menandai baris tertentu tanpa membatalkan keseluruhan transaksi.

Ketentuan yang dinilai: setiap respons non-2xx yang dapat dihasilkan service wajib dicantumkan dalam `openapi.yaml` beserta URI `type` dan satu contoh konkret. Kondisi error yang tidak terdokumentasi merupakan salah satu dari empat kelas cacat yang diperiksa pada latihan integrasi Pertemuan 7.

B.5 Versioning dan kebijakan kompatibilitas

Mata kuliah ini menggunakan *URI-path versioning*, yaitu penempatan versi pada URL server, sehingga setiap path diawali dengan `/v1/`.

Kriteria kompatibilitas: sebuah perubahan bersifat kompatibel apabila klien yang dibangun sebelum perubahan tersebut, dan tidak mengetahui adanya perubahan, tetap berfungsi tanpa modifikasi.

PERUBAHAN	STATUS DI V1	ALASAN
Menambahkan field request opsional	Diizinkan	Klien lama tidak mengirimkan field tersebut; server menerapkan nilai default.
Menambahkan field respons	Diizinkan	Klien lama mengabaikan field yang tidak dikenal. Ketentuan ini wajib dinyatakan dalam spesifikasi.
Menambahkan endpoint atau query parameter opsional	Diizinkan	Bersifat aditif.
Menambahkan nilai baru pada enumeration	Diizinkan bersyarat	Hanya apabila spesifikasi telah menetapkan cara klien menangani nilai enumeration yang tidak dikenal.
Mengubah field request opsional menjadi wajib	Tidak diizinkan	Seluruh pemanggil yang ada akan menerima <code>422</code> .
Menghapus atau mengganti nama field respons	Tidak diizinkan	Nama baru ditambahkan, nama lama dipertahankan, dan deprekasi dilakukan pada Pertemuan 8.
Mempersempit rentang nilai atau mengubah tipe	Tidak diizinkan	Request yang sebelumnya valid akan ditolak tanpa peringatan.
Mengubah makna field tanpa mengubah namanya	Tidak diizinkan	Tidak ada indikator perubahan pada protokol, sehingga kesalahan perhitungan pada klien tidak terdeteksi.

Serahkan dokumen satu halaman yang memuat: klasifikasi perubahan yang dianggap kompatibel, mekanisme pengumuman deprekasi menggunakan header `Deprecation` dan `Sunset` disertai catatan dalam spesifikasi, serta ketentuan penanganan field dan nilai enumeration yang tidak dikenal pada sisi klien.

B.6 Penyusunan dokumen

Langkah-langkah berikut dikerjakan secara berurutan. Penyusunan fragmen pertama memerlukan waktu sekitar empat puluh menit.

1. **Buat `openapi.yaml` yang hanya memuat header identifikasi.** Perlu diperhatikan bahwa `info.version` merupakan versi dokumen, yang berbeda dari `v1` pada URL.

```
openapi: 3.1.0
info:
  title: <Nama Sistem> API
  version: 0.1.0
  description: |
    <satu kalimat mengenai domain>.
    All monetary amounts are integers in the minor unit
    of the stated currency. Clients MUST ignore unknown
    response fields.
```

Dua kalimat terakhir pada `description` berfungsi mencegah kesalahan representasi nilai uang sebagai bilangan floating-point, dan menjadi dasar legalitas seluruh perubahan aditif pada bagian B.5.

2. **Pasang validator dan jalankan.**

```
npm init -y
npm install --save-dev @redocly/cli
npx redocly lint openapi.yaml
```

Hasil yang diharapkan: satu error yang menyatakan dokumen belum memiliki `paths`. Hasil ini menandakan validator telah berfungsi dan membaca berkas.

3. **Deklarasikan server** beserta versinya, sehingga path tidak perlu mengulang `/v1`.

```
servers:
  - url: https://api.contoh.example/v1
    description: Production
  - url: http://localhost:8080/v1
    description: Local development
```

4. **Definisikan schema pertama disertai `example` pada setiap field.** Schema ditempatkan di bawah `components` agar dapat direferensikan dari beberapa lokasi. Array `required` menentukan field yang wajib ada; field yang tidak tercantum di dalamnya harus dapat ditangani klien dalam kondisi tidak tersedia.
5. **Tulis operasi pertama.** Field `summary` memuat frasa singkat, sedangkan `description` memuat ketentuan yang berlaku pada operasi tersebut.

6. **Definisikan format error satu kali**, berupa schema `Problem` beserta response yang dapat digunakan kembali di bawah `components/responses`. Seluruh operasi berikutnya mereferensikan definisi tersebut.
7. **Modelkan resource utama beserta enumeration statusnya** secara eksplisit.

```
status:
  type: string
  enum: [placed, accepted, preparing, ready, collected, delivered, rejected, cancelled]
  description: |
    Clients MUST treat an unrecognised value as
    'in progress' and MUST NOT crash on it.
  example: placed
```

8. **Tulis operasi yang melibatkan transaksi keuangan beserta idempotency key-nya**. Header dideklarasikan sebagai parameter dengan `required: true`.

```
parameters:
  - name: Idempotency-Key
    in: header
    required: true
    schema: { type: string, format: uuid }
    description: |
      Generated by the client when the user confirms, and reused
      unchanged for every retry of that same intent. Retained 24
      hours. Reuse with a different body returns 409
      idempotency-key-reuse.
```

9. **Tinjau dokumen dari sudut pandang pembaca eksternal.**

```
npx redocly preview-docs openapi.yaml
```

Setiap operasi harus menampilkan ringkasan, contoh request, dan contoh respons. Contoh yang kosong menandakan `example` belum dicantumkan pada schema terkait.

10. **Jalankan mock server dari spesifikasi.**

```
npx @stoplight/prism-cli mock openapi.yaml
```

Kemudian, pada terminal kedua:

```
curl -i http://127.0.0.1:4010/<collection-anda>

curl -i -X POST http://127.0.0.1:4010/<koleksi-unsafe-anda> \
  -H 'Idempotency-Key: 0f7c1b9e-3d21-4a6f-9c05-8e2b7d41a9f0' \
  -H 'Content-Type: application/json' \
  -d '{ ... }'
```

Ulangi request tersebut tanpa header `Idempotency-Key`. Prism akan mengembalikan error karena dokumen mendeklarasikan header tersebut sebagai wajib. Dengan demikian, spesifikasi telah menegakkan ketentuan tanpa adanya kode implementasi.

Ketentuan penggunaan mock server. Mock server dipertahankan sepanjang semester. Mulai Pertemuan 3, anggota yang membangun klien menggunakan mock sampai service tersedia, dan selanjutnya harus dapat berpindah di antara keduanya hanya dengan mengubah base URL. Klien yang berfungsi terhadap implementasi tetapi tidak berfungsi terhadap mock menunjukkan adanya ketidaksesuaian antara implementasi dan spesifikasi.

B.7 Keluaran yang wajib tersedia

Berkas `openapi.yaml` di akar repositori yang memenuhi ketentuan berikut:

1. lolos `npx redocly lint` tanpa error;
2. mendefinisikan minimal tiga schema resource, masing-masing dengan array `required` dan `example` pada setiap properti;
3. mendefinisikan schema `Problem` beserta minimal satu response error yang dapat digunakan kembali;
4. menspesifikasikan satu operasi `GET` terhadap koleksi dengan parameter filter, dan satu operasi `POST` yang mewajibkan idempotency key;
5. menghasilkan respons yang berfungsi melalui `prism mock` dan didemonstrasikan secara langsung.

Dokumen tertulis dalam format Markdown pada direktori `docs/`, yang selanjutnya disusun menjadi PDF untuk A1:

1. **Tabel pemodelan resource** sesuai bagian B.1, memuat minimal dua kandidat yang ditolak;
2. **Katalog error**, memuat setiap URI `type` yang dapat dihasilkan service beserta status code, kondisi pemicu, extension members, dan tindakan klien yang berlaku;
3. **Pernyataan idempotency**, berupa empat ketentuan pada bagian B.3 yang diterapkan pada operasi unsafe kelompok masing-masing;
4. **Kebijakan kompatibilitas** sepanjang satu halaman sesuai bagian B.5;
5. **README**, memuat perintah untuk menjalankan mock server beserta tiga contoh perintah `curl`.

Kriteria penilaian Bagian B

KATEGORI KRITERIA

PASS Kelima ketentuan `openapi.yaml` terpenuhi dan didemonstrasikan sebelum sesi berakhir.

PARTIAL Dokumen lolos lint, tetapi mock server tidak menghasilkan contoh yang dapat digunakan, atau idempotency key tidak dispesifikasikan.

FAIL Kode implementasi service di-commit sebelum commit spesifikasi.

Kesalahan yang perlu dihindari

KESALAHAN	INDIKASI	PERBAIKAN
Spesifikasi disusun setelah kode	Schema mencerminkan struktur tabel basis data, termasuk kolom yang tidak diperlukan klien seperti <code>created_by_admin_id</code> . Indikasi lain berupa penggunaan <code>snake_case</code> pada nama field JSON.	Susun ulang schema berdasarkan domain.
Endpoint berbentuk layar	Endpoint dinamai menurut halaman antarmuka, misalnya <code>/home</code> , <code>/dashboard</code> , atau <code>/checkout-data</code> .	Uraikan menjadi resource penyusunnya sesuai bagian B.1.
Nilai uang sebagai bilangan desimal	<code>"total": 23000.0</code> , yang dapat menghasilkan <code>23000.000000001</code> .	Gunakan integer dalam satuan minor disertai field <code>currency</code> , dinyatakan pada <code>info.description</code> .
Status sebagai integer atau string bebas	<code>"state": 2</code> atau <code>"status": "Sedang disiapkan"</code> .	Gunakan enumeration string tertutup yang terdaftar dalam schema, disertai ketentuan penanganan nilai yang tidak dikenal.
Hanya jalur berhasil yang didokumentasikan	<code>responses</code> hanya memuat <code>200</code> .	Dokumentasikan setiap kegagalan yang dapat dihasilkan operasi, masing-masing dengan <code>type</code> dan contoh.
Idempotency key dinyatakan tanpa perincian	Tidak ada nama header, jendela retensi, maupun perilaku penggunaan ulang.	Terapkan empat ketentuan pada bagian B.3.
Timestamp tanpa zona waktu	<code>"placedAt": "2026-03-04 12:10:04"</code> .	Gunakan format RFC 3339 dengan offset: <code>2026-03-04T12:10:04+07:00</code> .
Koleksi tanpa batas jumlah	<code>GET /orders</code> tanpa <code>limit</code> dan tanpa cursor.	Spesifikasikan mekanisme paginasi pada tahap ini.
Commit dari akun bersama	Pernyataan kontribusi tidak dapat diverifikasi.	Setiap anggota melakukan commit dengan identitasnya sendiri.

Format dan pengumpulan

	BAGIAN A	BAGIAN B
Tag repositori	11	12
Jatuh tempo	Akhir sesi Pertemuan 2	Sebelum Pertemuan 3
Unit pengerjaan	Kelompok	Kelompok
Verifikasi	Dipresentasikan kepada demonstrator sebelum sesi berakhir	Mock server didemonstrasikan secara langsung

```
git tag l1 && git push --tags      # akhir Bagian A
git tag l2 && git push --tags      # akhir Bagian B
```

Tag menggunakan huruf L kecil diikuti angka.

Keterkaitan dengan A1

Dokumen yang dihasilkan pada tugas ini menjadi dasar penyusunan A1 — Interface Specification, dengan bobot 25% nilai akhir, yang dikumpulkan pada Pertemuan 4 dan direvisi pada Pertemuan 8.

BOBOT	KRITERIA A1	BAGIAN YANG DIKERJAKAN PADA TUGAS INI
30%	Pemodelan resource	B.1
25%	Kelengkapan dokumentasi jalur kegagalan	B.4
20%	Ketepatan semantik method, status code, dan idempotency	B.3
15%	Implementability: pihak eksternal dapat membangun klien tanpa pertanyaan lanjutan	B.6 langkah 9–10
10%	Kebijakan kompatibilitas	B.5

A1 dinilai satu kali berdasarkan versi revisi. Penilaian mencakup kualitas spesifikasi akhir dan kualitas tanggapan kelompok terhadap temuan pada latihan integrasi Pertemuan 7.

Soal pemeriksaan mandiri

1. Seorang anggota kelompok mengusulkan `POST /v1/orders/{id}/markReady`. Kemukakan dua alasan penolakan usulan tersebut, dan tuliskan URI penggantinya.
2. `PUT /v1/menu-items/itm_3Bn` dikirimkan dua kali akibat timeout. Tentukan apakah operasi tersebut safe dan apakah operasi tersebut idempoten, disertai justifikasi terpisah untuk masing-masing.
3. Sebuah klien melakukan retry atas sebuah order dan menerima `409 idempotency-key-reuse`. Tentukan kesalahan pada sisi klien dan lokasi kesalahan tersebut dalam kode klien.
4. Sebuah outlet kehabisan persediaan suatu item. Tentukan status code dan kategori kegagalan yang sesuai menurut bagian B.4, serta jelaskan konsekuensi penggunaan `500` dan `200` pada kondisi tersebut.
5. Field `grandTotal` akan diubah agar mencakup biaya pengiriman. Tentukan apakah perubahan tersebut kompatibel dalam `v1`. Apabila tidak, uraikan perubahan yang mencapai tujuan yang sama dan tetap kompatibel.